

COS40007

Portfolio Assessment 3

Studio 1 - 3

Duc Thuan Tran|104330455

Overview

This project developed an AI model to classify vegemite consistency levels using machine process data. After preprocessing the dataset (removing constant columns, converting low variance features to categorical, applying SMOTE for class balancing), multiple classification models were evaluated. The Random Forest classifier achieved superior performance with 99.4% accuracy on the balanced test set. Decision rules extracted from the model identified optimal machine setting ranges for each consistency level, translating complex AI insights into actionable production guidelines. The project demonstrates how machine learning can effectively optimize food manufacturing processes through accurate prediction and practical operational recommendations.

Source code:

https://github.com/ThuanDancoi/COS40007/blob/main/Portfolio%20Assessment%203/code/portfolio_assessment3.ipynb

Step1: Data Preparation

Initial Data Exploration and Understanding

The vegemite.csv dataset contains machine process and machine settings data related to vegemite production. Before applying any machine learning techniques, I conducted a thorough exploration of the dataset to understand its structure and characteristics.

The initial exploration revealed that the dataset contains 15,237 records with 47 features. These features represent various machine process parameters and settings that affect the consistency of vegemite production. The target variable 'Class' contains three values (0, 1, 2) representing different consistency levels of solids in vegemite production.

Class Distribution Analysis

The distribution of the target variable is a critical aspect of any classification problem. An initial analysis of the class distribution (Figure 1) revealed an imbalance:

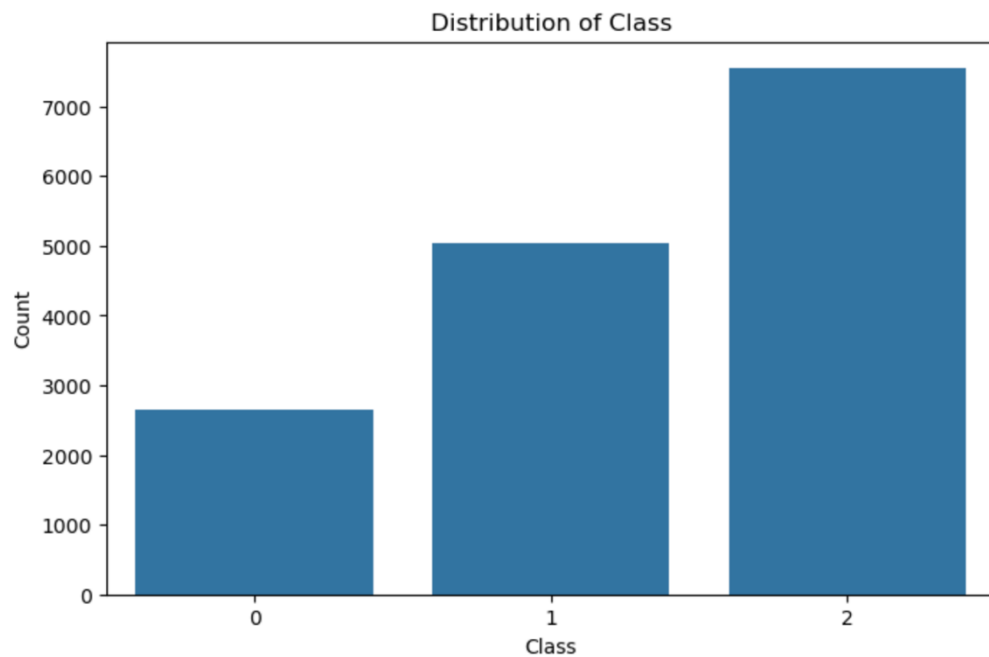


Image 1. Class Distribution

Class Distribution:

- Class 2: 7,548 samples (49.5%)
- Class 1: 5,047 samples (33.1%)
- Class 0: 2,642 samples (17.3%)

This distribution shows that Class 0 (representing one specific consistency level) is significantly underrepresented compared to the other classes. This imbalance could potentially bias our model toward the majority classes if not addressed appropriately.

Shuffling the Dataset

To ensure randomness in data selection, I first shuffled the entire dataset:

```
# 1. Shuffle the dataset
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
```

Image 2. Shuffle the dataset

Using a random state of 42 ensures reproducibility while maintaining the benefits of randomization.

Creating a Balanced Test Set

Following the assignment requirements, I extracted 1,000 data points for testing with a balanced distribution across classes:

```
# 2. Randomly take out 1000 data points with balanced classes
test_samples = []
class_distribution = {0:334, 1:333, 2:333}

for class_value, sample_count in class_distribution.items():
    class_rows = df[df['Class'] == class_value]

    if len(class_rows) < sample_count:
        print(f"Warning: Not enough samples for class {class_value}. Only {len(class_rows)} available.")
        sample_count = len(class_rows)

    selected = class_rows.sample(n=sample_count, random_state=42)
    test_samples.append(selected)
    print(f"Selected {sample_count} samples from class {class_value}")

    df = df[~df.index.isin(selected.index)]

test_df = pd.concat(test_samples).reset_index(drop=True)

train_df = df.reset_index(drop=True)
```

Image 3. Creating Balanced Test Set

The test dataset now contains:

- 334 samples from Class 0
- 333 samples from Class 1
- 333 samples from Class 2

This balanced distribution in the test set ensures fair evaluation across all classes. The remaining 14,237 data points form the training set.

Feature Construction and Cleaning

1. Removing Constant Value Columns

Constant value columns provide no discriminative information for classification and can be safely removed:

1. Checking for constant value columns

```
constant_cols = [col for col in train_df.columns if train_df[col].nunique() == 1]

if constant_cols:
    # Remove constant columns
    train_df = train_df.drop(columns=constant_cols)
    test_df = test_df.drop(columns=constant_cols)

constant_cols
```

```
['TFE Steam temperature SP', 'TFE Product out temperature']
```

Image 4. Removing Constant Value Columns

Two columns were identified as constant and removed:

- 'TFE Steam temperature SP'
- 'TFE Product out temperature'

2. Converting Columns with Few Integer Values to Categorical

Numeric columns with few unique values are often better treated as categorical features:

2. Checking for column with few integer values

```
: few_integer_cols = []
for col in train_df.columns:
    if train_df[col].dtype in ['int64', 'float64']:
        unique_values = train_df[col].nunique()
        if unique_values < 10 and col != 'Class':
            few_integer_cols.append(col)

# Convert integer columns to category
for col in few_integer_cols:
    train_df[col] = train_df[col].astype('category')
    test_df[col] = test_df[col].astype('category')
    print(f"Converted {col} to categorical")
```

```
Converted FFTE Feed tank level SP to categorical
Converted FFTE Pump 1 to categorical
Converted FFTE Pump 1 - 2 to categorical
Converted FFTE Pump 2 to categorical
Converted TFE Motor speed to categorical
```

Image 5. Converting columns with few integer values to categorical

Five columns were converted to categorical type:

- FFTE Feed tank level SP (3 unique values)
- FFTE Pump 1 (5 unique values)
- FFTE Pump 1 - 2 (4 unique values)
- FFTE Pump 2 (5 unique values)
- TFE Motor speed (3 unique values)

3. Training Set Class Distribution:



Image 6. Class distribution in training data

- Class 2: 7,215 samples
- Class 1: 4,714 samples
- Class 0: 2,308 samples

To quantify this imbalance, I calculated the imbalance ratio:

```
# Choosing the methods base on the ratio performance
min_class_count = class_counts.min()
max_class_count = class_counts.max()
imbalance_ratio = min_class_count / max_class_count

print(f"Imbalance ratio (min/max): {imbalance_ratio:.4f}")

BALANCED_THRESHOLD = 0.8
MODERATE_IMBALANCE = 0.5
SEVERE_IMBALANCE = 0.2

if imbalance_ratio >= BALANCED_THRESHOLD:
    print("Classes are fairly balanced. No resampling needed.")
    resampling_method = None
elif imbalance_ratio >= MODERATE_IMBALANCE:
    print("Moderate class imbalance detected. Will use class weights in model training.")
    class_weights = {i: len(train_df) / (len(np.unique(train_df['Class'])) * count)
                     for i, count in enumerate(class_counts)}
    print(f"Class weights: {class_weights}")
    resampling_method = "class_weights"
elif imbalance_ratio >= SEVERE_IMBALANCE:
    print("Significant class imbalance detected. Will apply SMOTE oversampling.")
    resampling_method = "smote"
else:
    print("Severe class imbalance detected. Will apply combination of undersampling and SMOTE.")
    resampling_method = "undersampling+smote"
```

Imbalance ratio (min/max): 0.3199
Significant class imbalance detected. Will apply SMOTE oversampling.

Image 7. Imbalance ratio calculation

With an imbalance ratio of 0.3199, the SMOTE (Synthetic Minority Over-sampling Technique) method was selected to address the class imbalance.

SMOTE

```

: X = train_df.drop('Class', axis=1)
  y = train_df['Class']

print("Class distribution before resampling:")
print(y.value_counts())

# Apply SMOTE
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Create a new DataFrame with the resampled data
resampled_df = pd.DataFrame(X_resampled, columns=X.columns)
resampled_df['Class'] = y_resampled

print("Class distribution after resampling:")
print(pd.Series(y_resampled).value_counts())

train_df = resampled_df

Class distribution before resampling:
Class
2    7215
1    4714
0    2308
Name: count, dtype: int64
Class distribution after resampling:
Class
2    7215
0    7215
1    7215
Name: count, dtype: int64

```

Image 8. Smote applying for class imbalance

After applying SMOTE, the training dataset had a perfectly balanced class distribution with 7,215 samples in each class.

By systematically addressing data preparation requirements, I created a solid foundation for feature selection and model training in subsequent steps. The balanced test set will provide a fair evaluation of model performance, while the SMOTE-balanced training set will help prevent bias toward majority classes during model training.

4. Exploring Composite Features Through Correlation Analysis

To identify potentially useful composite features, I conducted a correlation analysis to understand relationships between variables. This approach helps identify features that might work well together or could be combined to create more informative features.

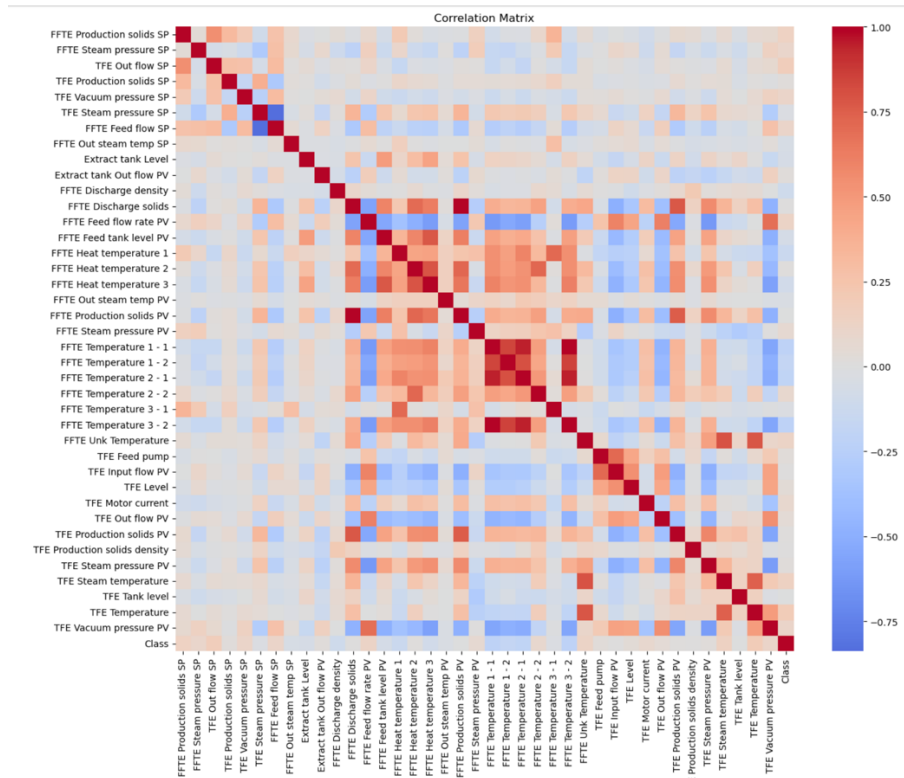


Image 9. Correlation Analysis

The correlation matrix (shown in Image 3) revealed several interesting patterns:

- Strong correlations between certain set points (SP) and their corresponding process variables (PV)
- Moderate correlations between temperature measurements at different points in the process
- Notable correlations between feed flow and pressure parameters

Based on these observations, I created three types of composite features:

- **Temperature Averaging:**

Several temperature sensors showed high correlation, suggesting they measure related aspects of the process. Averaging these readings provides a more robust temperature indicator:

```
# Average temperature
train_df['FFTE_avg_temperature'] = (train_df['FFTE Temperature 1 - 1'] +
                                     train_df['FFTE Temperature 1 - 2'] +
                                     train_df['FFTE Temperature 2 - 1'] +
                                     train_df['FFTE Temperature 3 - 2']) / 4
```

Image 10. Temperature Averaging

- **Flow-Pressure Ratio:**

The correlation between feed flow and steam pressure suggested an important relationship. Creating a ratio feature captures this relationship:

```
# Ratio between feed flow and steam pressure setpoints
train_df['flow_pressure_ratio'] = train_df['FFTE Feed flow SP'] / train_df['FFTE Steam pressure SP']
```

Image 11. Flow-Pressure Ratio

- **Setpoint-Process Variable Deviations:**

The difference between setpoints (target values) and process variables (actual values) can indicate how well the process is controlled:

```
# Deviation between actual and target solids
train_df['FFTE_Production_solids_deviation'] = train_df['FFTE Production solids PV'] - train_df['FFTE Production solids SP']
```

Image 12. Setpoint-process variable deviations

5. Final Feature Count

After completing all the data preparation steps, I tallied the final feature count:

5. Checking the final features count

```
num_features = len([col for col in train_df.columns if col != 'Class'])
print(num_features)
```

47

Image 13. Final features count

The final dataset contains 47 features (excluding the target 'Class' column). This count includes:

- Original features remaining after removing constant columns
- Categorical features converted from numeric columns with few integer values
- Newly created composite features based on correlation analysis

The feature count suggests a moderately complex dataset with sufficient information for model training while not being excessively high-dimensional.

Step2: Feature Selection, Model Training and Evaluation

Feature Selection with SelectKBest

After preparing the dataset in Step 1, I proceeded to select the most relevant features for model training. Feature selection is crucial for improving model performance, reducing overfitting, and decreasing computational complexity.

I implemented a systematic approach using the SelectKBest method with the `f_classif` scoring function, which employs ANOVA F-value to identify features with the strongest relationship to the target variable.

6. Feature Selection with SelectKBest

```
X_train = train_df.drop("Class", axis=1)
y_train = train_df["Class"]
X_test = test_df.drop("Class", axis=1)
y_test = test_df["Class"]

X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)

missing_cols = set(X_train.columns) - set(X_test.columns)
for col in missing_cols:
    X_test[col] = 0

selector = SelectKBest(f_classif, k='all')
selector.fit(X_train, y_train)

feature_scores = pd.DataFrame({
    'Feature': X_train.columns,
    'Score' : selector.scores_
})

feature_scores = feature_scores.sort_values('Score', ascending = False)
```

Image 14. Feature Selection with SelectKBest

After analyzing the feature importance scores, I selected the top 20 features that collectively provided the most discriminative power for the classification task:

```
top_features = feature_scores.head(20)['Feature'].values
X_train_selected = X_train[top_features]
X_test_selected = X_test[top_features]
```

Image 15. Select 20 best features base on the score.

Model Training and Evaluation

With the selected features, I trained multiple machine learning models to compare their performance. As required by the assignment, I included DecisionTreeClassifier and added four other models to provide a comprehensive comparison:

```

# Model training and evaluation
models = {
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'SVM': SVC(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5)
}

results = {}
model_performances = {}

for name, model in models.items():
    model.fit(X_train_selected, y_train)

    y_pred = model.predict(X_test_selected)

    # Evaluate model
    accuracy = accuracy_score(y_test, y_pred)
    report = classification_report(y_test, y_pred, output_dict=True)
    conf_matrix = confusion_matrix(y_test, y_pred)

    # Store results
    results[name] = {
        'model': model,
        'accuracy': accuracy,
        'report': report,
        'confusion_matrix': conf_matrix
    }

    # Store performance metrics for comparison
    model_performances[name] = {
        'Accuracy': accuracy,
        'Precision (macro avg)': report['macro avg']['precision'],
        'Recall (macro avg)': report['macro avg']['recall'],
        'F1-score (macro avg)': report['macro avg']['f1-score']
    }

print(f"\nModel: {name}")
print(f"Accuracy: {accuracy:.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred))
print("Confusion Matrix:")
print(conf_matrix)

```

Image 16. Model Training & Model Evaluating

Each model was evaluated using multiple metrics to provide a comprehensive assessment of their performance:

- **Accuracy:** The proportion of correct predictions among the total number of predictions.
- **Precision:** The ability of the model to avoid false positives (proportion of true positives among all positive predictions).
- **Recall:** The ability of the model to find all positive samples (proportion of true positives among all actual positives).
- **F1-score:** The harmonic mean of precision and recall, providing a balance between the two metrics.
- **Confusion Matrix:** A table showing the number of true positives, false positives, true negatives, and false negatives for each class.

Model Comparison

To facilitate comparison between models, I created a comparison table with all performance metrics. The model comparison yielded the following results:

Model	Accuracy	Precision	Recall	F1-score
Decision Tree	0.840	0.850	0.840	0.839
Random Forest	0.994	0.994	0.994	0.994

Gradient Boosting	0.882	0.881	0.881	0.881
SVM	0.473	0.576	0.472	0.396
KNN	0.903	0.902	0.902	0.902

Best Model Selection

After comparing all models across different evaluation metrics, I selected the best-performing model based on the F1-score, which provides a balanced measure of precision and recall. The Random Forest classifier emerged as the best-performing model with an impressive F1-score of 0.9940. This model demonstrated exceptional performance across all metrics:

Best performing model: Random Forest with F1-score: 0.9940

Image 17. Best Model Result

Model Saving

To preserve the best model for future use, I saved it using the pickle library

Saved best model (Random Forest) to 'best_model.pkl'

Image 18. Model Saving

Step 3: ML to AI

In this step, I transformed the trained machine learning model into a functioning AI system by deploying it to make predictions on new, unseen data. This process involved loading the saved model, preparing the test data, making predictions, and evaluating the model's performance in a real-world context. First, I loaded the best-performing model (Random Forest) that was saved during Step 2. The output confirmed successful loading of the RandomForestClassifier with the correct parameters:

12. Take the unused 1000 rows & 13. Load the Model

```
with open('best_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)
loaded_model
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Image 19. Load the model

To ensure consistent processing, I prepared the test data with the same feature format used during training. With the model loaded and test data prepared, I made predictions on the 1,000 unseen test samples

14. Formatting the test data with the same features as training

```
X_test = test_df.drop('Class', axis=1)
y_test = test_df['Class']

X_test = pd.get_dummies(X_test, drop_first=True)

missing_cols = set(X_train.columns) - set(X_test.columns)
for col in missing_cols:
    X_test[col] = 0
```

15. Make predictions using loaded model & 16. Measure performance

```
y_pred = loaded_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print("\nClassification Report:")
print(report)
print("\nConfusion Matrix:")
print(conf_matrix)
```

Image 20. Formatting Test as training, Make prediction, Measure Performance

The model achieved outstanding performance on the unseen test data:

```
Classification Report:
              precision    recall  f1-score   support

     0       1.00      0.99      0.99        334
     1       0.99      1.00      0.99        333
     2       1.00      0.99      1.00        333

 accuracy      0.99
 macro avg     0.99
weighted avg     0.99
```

```
Confusion Matrix:
[[331  2  1]
 [ 1 332  0]
 [ 0  2 331]]
```

Image 21. Classification report and Confusion Matrix

I visualized the confusion matrix to provide a clearer understanding of the model's performance across different classes:

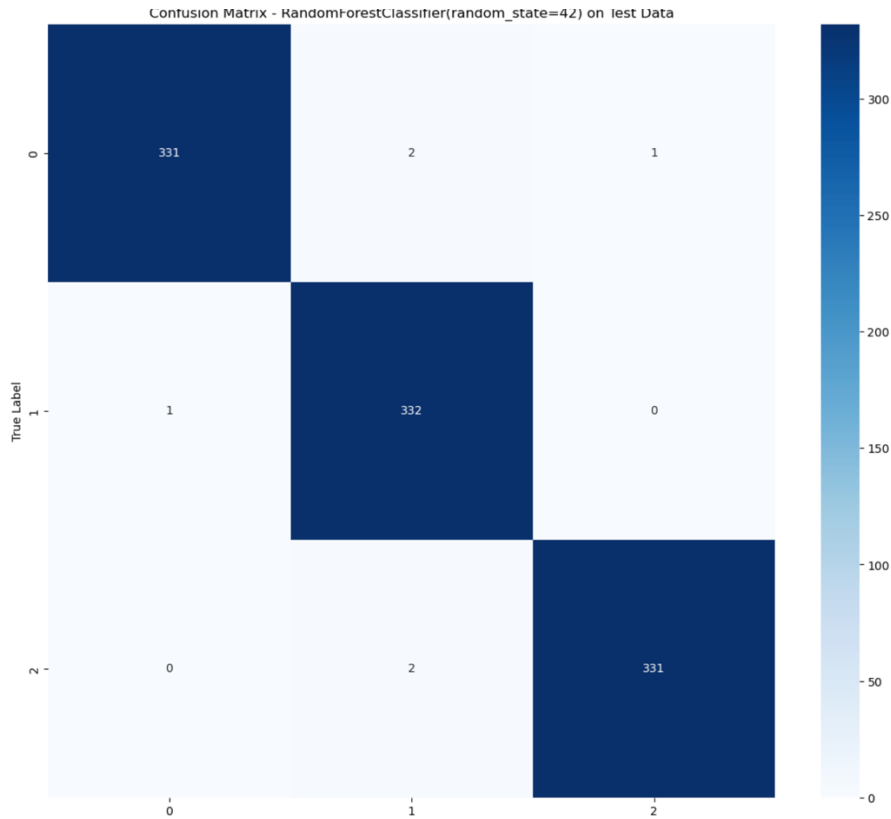


Image 22. Confusion Matrix Heat Map

The visualization (Image 2) shows that the model correctly classified most samples, with only 6 misclassifications out of 1,000 test samples. This represents a 99.4% accuracy rate on completely unseen data, demonstrating the model's strong generalization capabilities.

Evaluating All Models on the Test Set

To validate our model selection and ensure the Random Forest remained the best choice, I evaluated all trained models on the test dataset:

```
all_model_test_performance = {}
for name, model_info in results.items():
    model = model_info['model']
    test_pred = model.predict(X_test)
    test_acc = accuracy_score(y_test, test_pred)
    test_class_report = classification_report(y_test, test_pred, output_dict=True)

    all_model_test_performance[name] = {
        'Test Accuracy': test_acc,
        'Test Precision': test_class_report['macro avg']['precision'],
        'Test Recall': test_class_report['macro avg']['recall'],
        'Test F1-score': test_class_report['macro avg']['f1-score']
    }
test_comparison_df = pd.DataFrame(all_model_test_performance).T
print("\nModel Comparison on Test Dataset:")
print(test_comparison_df)
```

```
Model Comparison on Test Dataset:
              Test Accuracy  Test Precision  Test Recall  Test F1-score
Decision Tree           0.840         0.850074      0.840131      0.839454
Random Forest           0.994         0.994024      0.994003      0.994003
Gradient Boosting       0.882         0.881867      0.881972      0.881892
SVM                     0.473         0.576320      0.472676      0.396542
KNN                     0.903         0.902956      0.902978      0.902881
```

Image 22. Evaluating the best model on test set

Model	Accuracy	Precision	Recall	F1-score
-------	----------	-----------	--------	----------

Decision Tree	0.840	0.850	0.840	0.839
Random Forest	0.994	0.994	0.994	0.994
Gradient Boosting	0.882	0.881	0.881	0.881
SVM	0.473	0.576	0.472	0.396
KNN	0.903	0.902	0.902	0.902

Base on the table output: Random Forest is the best model on the test set

Step 4: Develop rules from the ML model

The final step of this project involved extracting actionable rules from the machine learning model to guide vegemite production parameters. This step is particularly valuable as it transforms the "black box" nature of complex ML models into clear, interpretable guidelines that can be directly implemented by production staff.

Extracting Set Point (SP) Features

First, I focused exclusively on the set point (SP) features, which represent the controllable parameters in vegemite production:

```
sp_features = [col for col in train_df.columns if col.endswith('SP') and col != 'Class']

sp_features

['FFTE Feed tank level SP',
 'FFTE Production solids SP',
 'FFTE Steam pressure SP',
 'TFE Out flow SP',
 'TFE Production solids SP',
 'TFE Vacuum pressure SP',
 'TFE Steam pressure SP',
 'FFTE Feed flow SP',
 'FFTE Out steam temp SP']
```

Image 23. Extracting Set Point (SP) Features

Building a Decision Tree Model with SP Features Only

Using only these controllable parameters, I created a dataset focused on SP features and trained a decision tree model:

```
X_sp_df = train_df[sp_features]
y_sp_df = train_df['Class']

X_sp_encoded_df = pd.get_dummies(X_sp_df, drop_first=True)

X_sp_df = X_sp_encoded_df

dt_model = DecisionTreeClassifier(random_state=42, max_depth=5)
dt_model = dt_model.fit(X_sp_df, y_sp_df)

tree_rules = export_text(dt_model, feature_names=list(X_sp_df.columns))
print("\nDecision Tree Rules: ")
print(tree_rules)
```

Image 24. Building a Decision Tree Model with SP Features Only

I intentionally limited the tree depth to 5 to create rules that are both interpretable and practical. Deeper trees might provide higher accuracy but would produce overly complex rules that would be difficult to implement in a production environment.

Analyzing the Decision Rules

Decision Tree Rules:

```

|--- FFTE Feed flow SP <= 10198.77
|   |--- FFTE Feed flow SP <= 9231.11
|   |   |--- TFE Out flow SP <= 2173.78
|   |   |   |--- FFTE Feed tank level SP_50.0 <= 0.50
|   |   |   |   |--- TFE Production solids SP <= 68.69
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- TFE Production solids SP > 68.69
|   |   |   |   |   |--- class: 1
|   |   |   |--- FFTE Feed tank level SP_50.0 > 0.50
|   |   |   |   |--- FFTE Feed flow SP <= 9200.68
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- FFTE Feed flow SP > 9200.68
|   |   |   |   |   |--- class: 1
|   |   |--- TFE Out flow SP > 2173.78
|   |   |   |--- FFTE Production solids SP <= 41.31
|   |   |   |   |--- TFE Out flow SP <= 2507.03
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- TFE Out flow SP > 2507.03
|   |   |   |   |   |--- class: 1
|   |   |   |--- FFTE Production solids SP > 41.31
|   |   |   |   |--- TFE Production solids SP <= 68.95
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- TFE Production solids SP > 68.95
|   |   |   |   |   |--- class: 1
|   |--- FFTE Feed flow SP > 9231.11
|   |   |--- FFTE Steam pressure SP <= 128.52
|   |   |   |--- FFTE Out steam temp SP <= 49.28
|   |   |   |   |--- FFTE Production solids SP <= 40.50
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- FFTE Production solids SP > 40.50
|   |   |   |   |   |--- class: 1
|   |   |   |--- FFTE Out steam temp SP > 49.28
|   |   |   |   |--- TFE Out flow SP <= 2067.13
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- TFE Out flow SP > 2067.13
|   |   |   |   |   |--- class: 0
|   |   |--- FFTE Steam pressure SP > 128.52
|   |   |   |--- TFE Steam pressure SP <= 84.07
|   |   |   |   |--- FFTE Production solids SP <= 40.11
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- FFTE Production solids SP > 40.11
|   |   |   |   |   |--- class: 0
|   |   |   |--- TFE Steam pressure SP > 84.07
|   |   |   |   |--- FFTE Feed flow SP <= 9499.99
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- FFTE Feed flow SP > 9499.99
|   |   |   |   |   |--- class: 2
|--- FFTE Feed flow SP > 10198.77

```

Image 25. Decision Tree result part 1

```

|--- FFTE Feed flow SP > 10198.77
|   |--- TFE Production solids SP <= 84.60
|   |   |--- FFTE Out steam temp SP <= 50.14
|   |   |   |--- TFE Production solids SP <= 66.99
|   |   |   |   |--- FFTE Steam pressure SP <= 117.03
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- FFTE Steam pressure SP > 117.03
|   |   |   |   |   |--- class: 2
|   |   |   |--- TFE Production solids SP > 66.99
|   |   |   |   |--- TFE Production solids SP <= 78.09
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- TFE Production solids SP > 78.09
|   |   |   |   |   |--- class: 1
|   |   |--- FFTE Out steam temp SP > 50.14
|   |   |   |--- FFTE Out steam temp SP <= 51.10
|   |   |   |   |--- FFTE Steam pressure SP <= 127.64
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- FFTE Steam pressure SP > 127.64
|   |   |   |   |   |--- class: 1
|   |   |--- FFTE Out steam temp SP > 51.10
|   |   |   |--- FFTE Feed flow SP <= 10299.85
|   |   |   |   |--- class: 2
|   |   |   |--- FFTE Feed flow SP > 10299.85
|   |   |   |   |--- class: 2
|   |--- TFE Production solids SP > 84.60
|   |   |--- FFTE Feed tank level SP_50.0 <= 0.50
|   |   |   |--- TFE Vacuum pressure SP <= -40.68
|   |   |   |   |--- TFE Vacuum pressure SP <= -63.66
|   |   |   |   |   |--- class: 0
|   |   |   |   |--- TFE Vacuum pressure SP > -63.66
|   |   |   |   |   |--- class: 0
|   |   |   |--- TFE Vacuum pressure SP > -40.68
|   |   |   |   |--- class: 2
|   |   |--- FFTE Feed tank level SP_50.0 > 0.50
|   |   |   |--- TFE Vacuum pressure SP <= -47.44
|   |   |   |   |--- class: 2
|   |   |   |--- TFE Vacuum pressure SP > -47.44
|   |   |   |   |--- TFE Steam pressure SP <= 115.00
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- TFE Steam pressure SP > 115.00
|   |   |   |   |   |--- class: 1

```

Image 26. Decision Tree result part 2

The resulting decision tree provides clear pathways from set point configurations to expected consistency classes. From the tree structure, I extracted the following key rules for each class:

For Class 0 (Low consistency):

- If FFTE Feed flow SP > 9231.11 AND FFTE Steam pressure SP > 128.52 AND TFE Steam pressure SP ≤ 84.07 AND FFTE Production solids SP > 40.11
- If FFTE Feed flow SP ≤ 9231.11 AND TFE Out flow SP > 2173.78 AND FFTE Production solids SP > 41.31 AND TFE Production solids SP ≤ 68.95
- If FFTE Feed flow SP > 10198.77 AND TFE Production solids SP ≤ 84.60 AND FFTE Out steam temp SP ≤ 50.14 AND TFE Production solids SP ≤ 66.99 AND FFTE Steam pressure SP ≤ 117.03

For Class 1 (Medium consistency):

- If FFTE Feed flow SP ≤ 9231.11 AND TFE Out flow SP ≤ 2173.78 AND FFTE Feed tank level SP ≤ 0.50 AND TFE Production solids SP ≤ 68.69
- If FFTE Feed flow SP ≤ 9231.11 AND TFE Out flow SP > 2173.78 AND FFTE Production solids SP ≤ 41.31 AND TFE Out flow SP > 2507.03
- If FFTE Feed flow SP > 9231.11 AND FFTE Steam pressure SP ≤ 128.52 AND FFTE Out steam temp SP > 49.28 AND TFE Out flow SP ≤ 2067.13

For Class 2 (High consistency):

- If FFTE Feed flow SP ≤ 9231.11 AND TFE Out flow SP ≤ 2173.78 AND FFTE Feed tank level SP > 0.50 AND FFTE Feed flow SP ≤ 9200.68
- If FFTE Feed flow SP > 9231.11 AND FFTE Steam pressure SP ≤ 128.52 AND FFTE Out steam temp SP ≤ 49.28 AND FFTE Production solids SP ≤ 40.50
- If FFTE Feed flow SP > 10198.77 AND TFE Production solids SP ≤ 84.60 AND FFTE Out steam temp SP ≤ 50.14 AND TFE Production solids SP ≤ 66.99 AND FFTE Steam pressure SP > 117.03

Simplified Production Guidelines

Based on these decision rules, I formulated the following simplified guidelines for process operators:

For Class 0 (Low consistency) production:

- Keep FFTE Production solids SP above 40.11 but below 41.31
- Maintain TFE Production solids SP below 68.95
- Set FFTE Steam pressure SP below 117.03
- Target FFTE Feed flow SP above 9231.11 but below 10198.77

For Class 1 (Medium consistency) production:

- Set TFE Production solids SP between 68.69 and 78.09
- Maintain FFTE Feed tank level SP below 0.50
- Keep TFE Out flow SP above 2507.03 when FFTE Production solids SP is low
- Set FFTE Out steam temp SP between 49.28 and 51.10

For Class 2 (High consistency) production:

- Set FFTE Feed tank level SP above 0.50
- Keep FFTE Feed flow SP below 9200.68
- Maintain FFTE Production solids SP below 40.50
- Set FFTE Steam pressure SP above 117.03 but below 127.64
- Keep TFE Vacuum pressure SP above -47.44

Appendix: Source Code Repository

The complete source code for this project is available at the following link:

https://github.com/ThuanDancoi/COS40007/blob/main/Portfolio%20Assessment%203/code/portfolio_assessment3.ipynb

References

https://scikit-learn.org/stable/modules/model_evaluation.html

https://scikit-learn.org/stable/modules/model_evaluation.html#classification-report

https://scikit-learn.org/stable/modules/model_evaluation.html#confusion-matrix

<https://datasciencehorizons.com/handling-imbalanced-datasets-in-scikit-learn-techniques-and-best-practices/>

https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.mutual_info_classif.html#sklearn.feature_selection.mutual_info_classif

<https://www.datacamp.com/tutorial/random-forests-classifier-python>

<https://medium.com/intuition/gradient-descent-visualization-285d3dd0fe00>